

Modelación y Diseño de Sistemas

2026 · UNC-ESCMB-AUSI

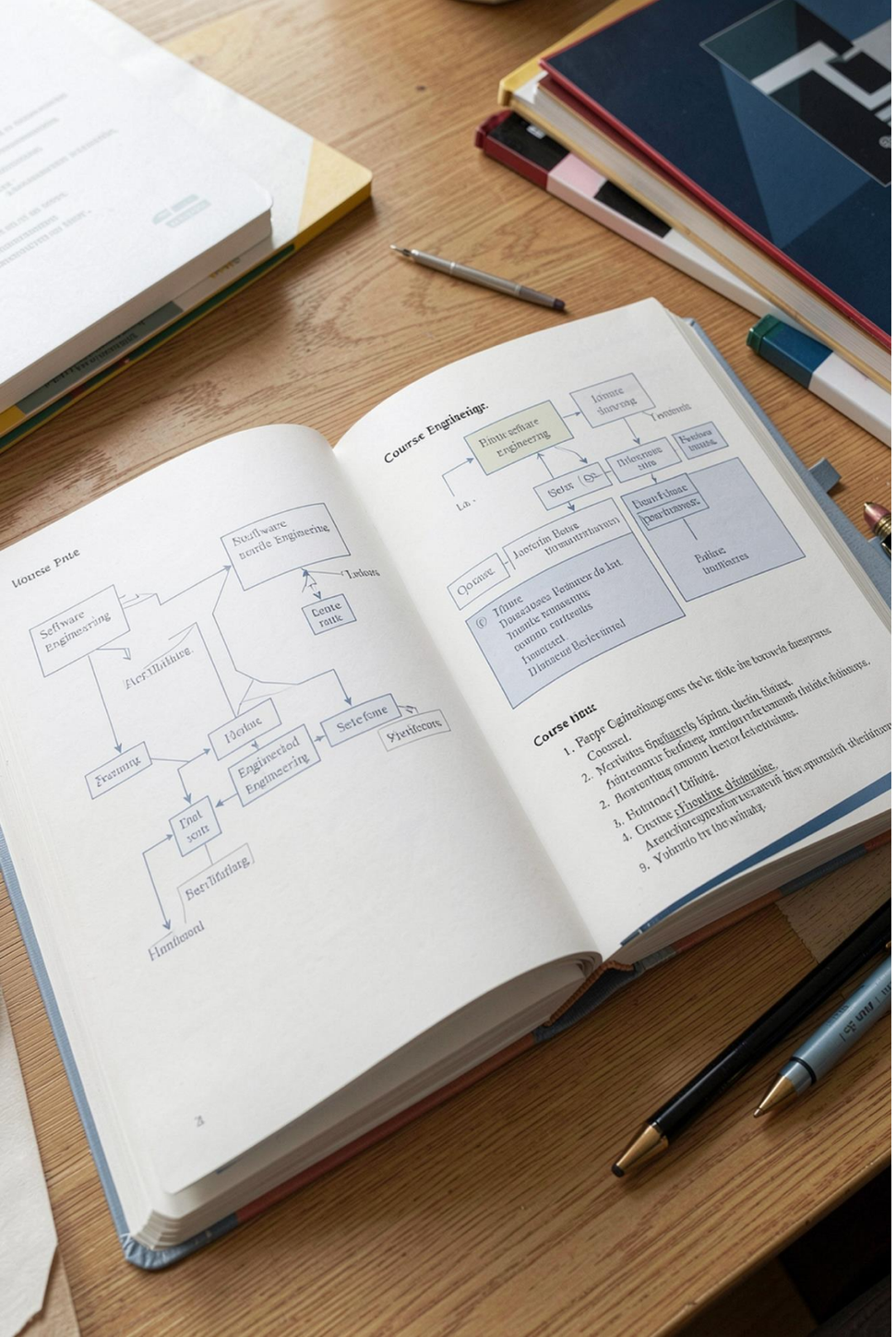
Clase 1 · Introducción

Ing. Fernando Bono

linkedin.com/in/fernandobono/

Bienvenidos

Esta materia aborda los fundamentos del modelado y diseño de sistemas de software, desde la visión general hasta las metodologías ágiles modernas.



Contenido del Curso

1

Unidad I

Software: Visión General

Introducción al modelado de software

2

Unidad II

Modelado de Software

Casos de uso · Notación UML

3

Unidad III

Proyectos, Metodologías y Marcos de Trabajo

Ciclo de Vida · Arquitectura · Metodologías Ágiles

MyD 2026 · Ing. Fernando Bono

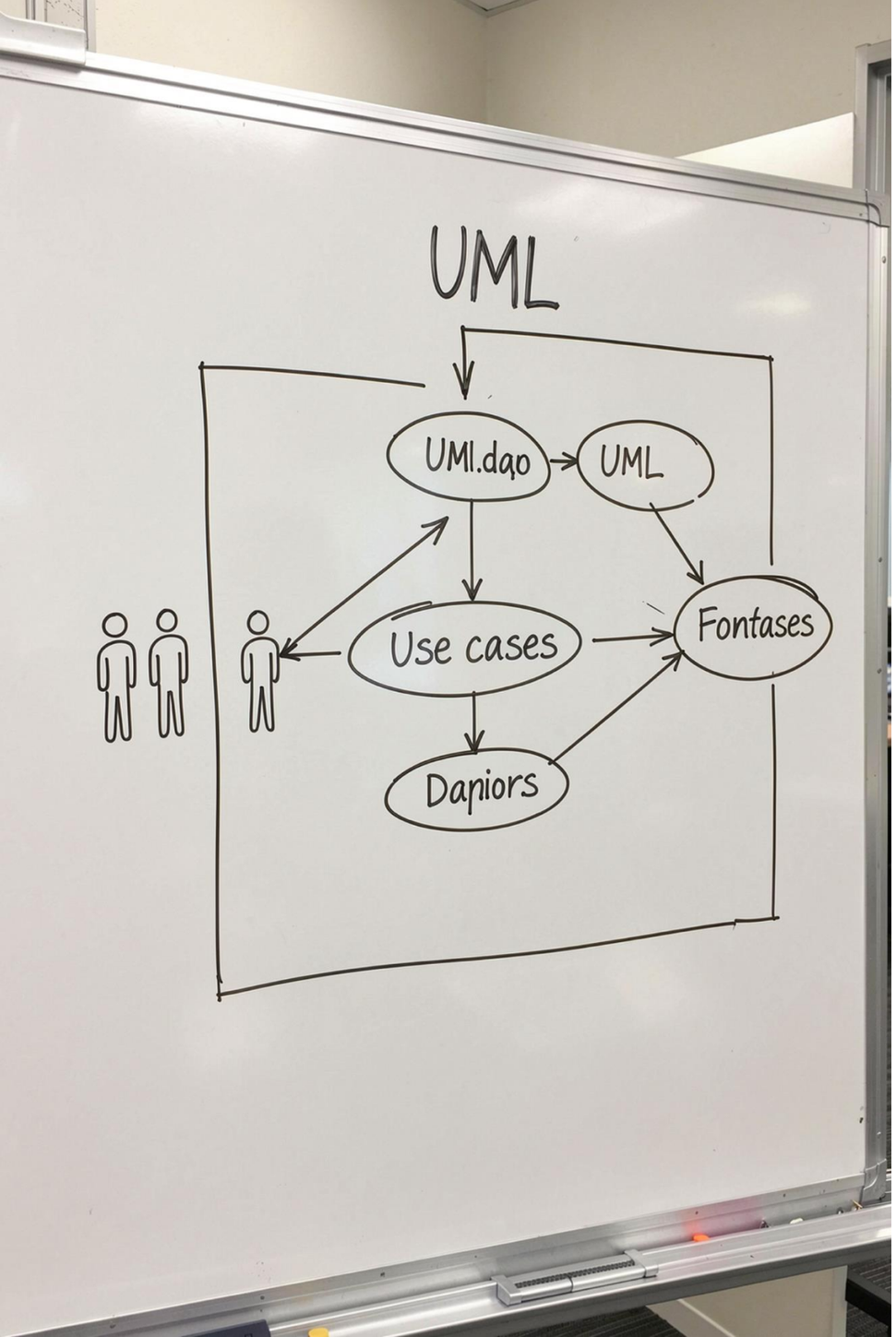
Contenido Detallado — Unidad I y II

El Modelado de Software

- Modelado orientado a Objetos y el UML
- Diseño de Arquitectura de Software
- Método y Notación
- COMET: Metodología colaborativa
- El estándar UML
- Evolución del modelado de software
- Métodos de Diseño y Análisis OO
- Métodos Concurrentes, distribuidos y en tiempo real

Unidad II — Modelado de Software

- Modelado de Requerimientos
- Casos de uso y Actores
- Identificación de casos de uso
- Documentación en el Modelo de Caso de Uso
- Requerimientos No funcionales
- Diagramas de actividades
- Notación UML completa



Modelado de Requerimientos

UNIDAD II · MODELANDO CASOS DE USO

Casos de Uso

Identificación, documentación y modelado de casos de uso en el sistema

Actores

Identificación de actores que interactúan con el sistema

Requerimientos No Funcionales

Restricciones de tiempo, proceso y estándares del sistema

Diagramas de Actividades

Descripción de la lógica de procesos y workflows

MyD 2026 · Ing. Fernando Bono

Diagramas UML

UNIDAD II · NOTACIÓN UML



Diagramas de Casos de Uso

Captura de requerimientos funcionales del sistema



Clase y Objetos

Diagramas de Clases con propiedades y operaciones



Diagramas de Interacción

Colaboración entre objetos para producir comportamiento



State Machine

Comportamiento de un objeto a lo largo de su vida



Packages

Agrupación de elementos para descripción de grandes sistemas

Ciclo de Vida del Software

UNIDAD III · PROYECTOS, METODOLOGÍAS Y MARCOS DE TRABAJO

1

Modelos de Ciclos de Vida

Diferentes enfoques para estructurar el desarrollo de software

2

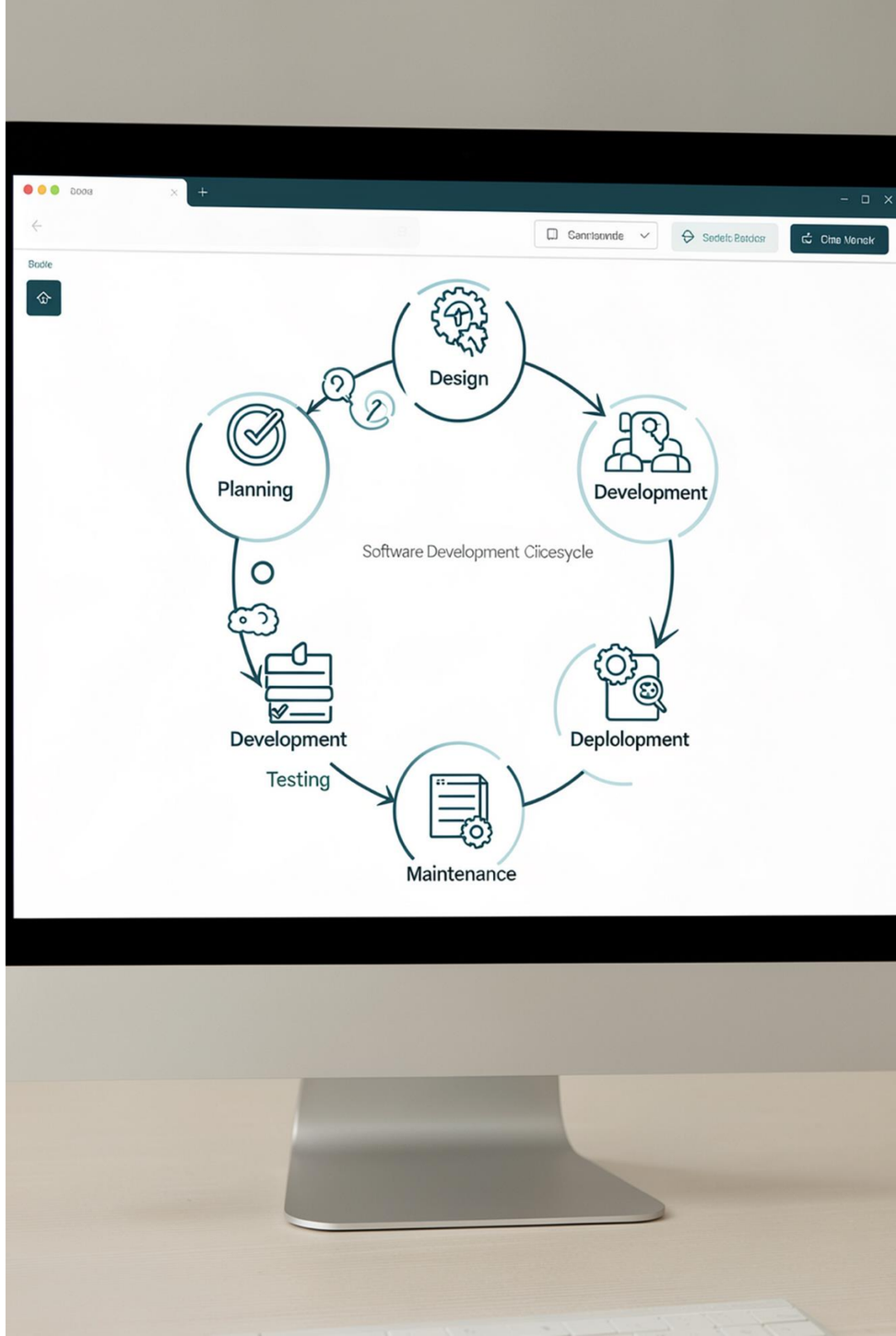
Diseño, Verificación y Validación

Asegurar que el software cumple con los requerimientos

3

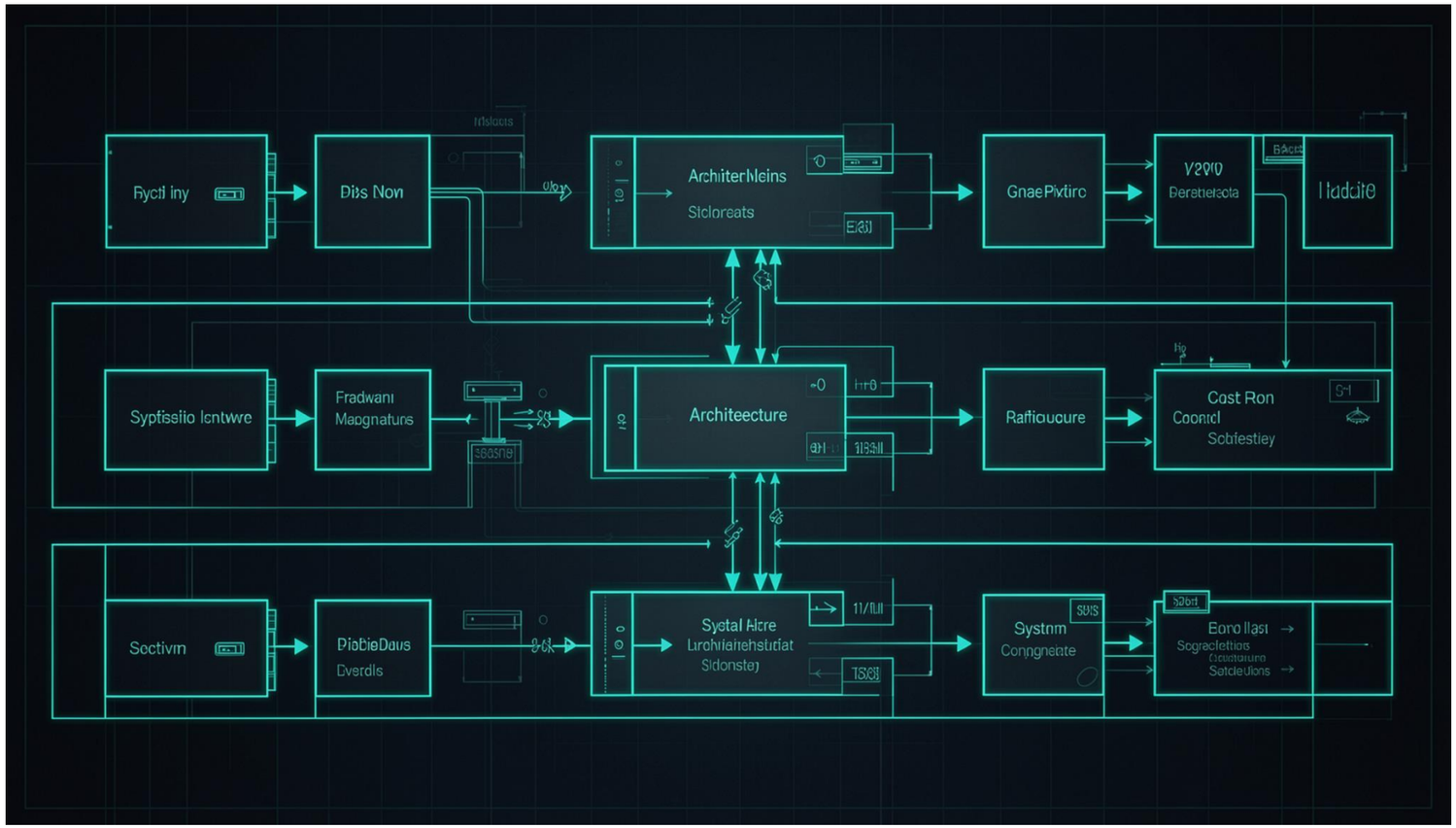
Testing de Software

Técnicas y estrategias para garantizar la calidad del producto



Diseño de Software y Conceptos de Arquitectura

UNIDAD III · PROYECTOS, METODOLOGÍAS Y MARCOS DE TRABAJO



Temas Clave

- Conceptos de Orientación a Objetos
- Ocultamiento de información
- Herencia
- Procesamiento concurrente
- Patrones de Diseño
- Arquitectura de Software y componentes
- Atributos de Calidad de Software



Metodologías Ágiles

UNIDAD III · EL MANIFIESTO AGILE

Orígenes

Historia y contexto del surgimiento del Manifiesto Agile

Utilización

Aplicación práctica en proyectos de software modernos

Características

Principios fundamentales que definen el enfoque ágil

Valores del Manifiesto

Los cuatro valores centrales que guían el desarrollo ágil

Scrum Framework

UNIDAD III · METODOLOGÍAS ÁGILES



Nivel de Ruido en el Proyecto

El Scrum Framework organiza el trabajo en ciclos cortos llamados **Sprints**, con ceremonias regulares para mantener al equipo alineado y enfocado.

El Burndown Chart permite visualizar el progreso del equipo durante el Sprint y anticipar desviaciones respecto al plan.

Modelación y Diseño de Sistemas

Analista Universitario de Sistemas Informáticos

UNC · ESCMB · AUSI · 2026

MyD 2026 · Ing. Fernando Bono



¿Qué es un Modelo?

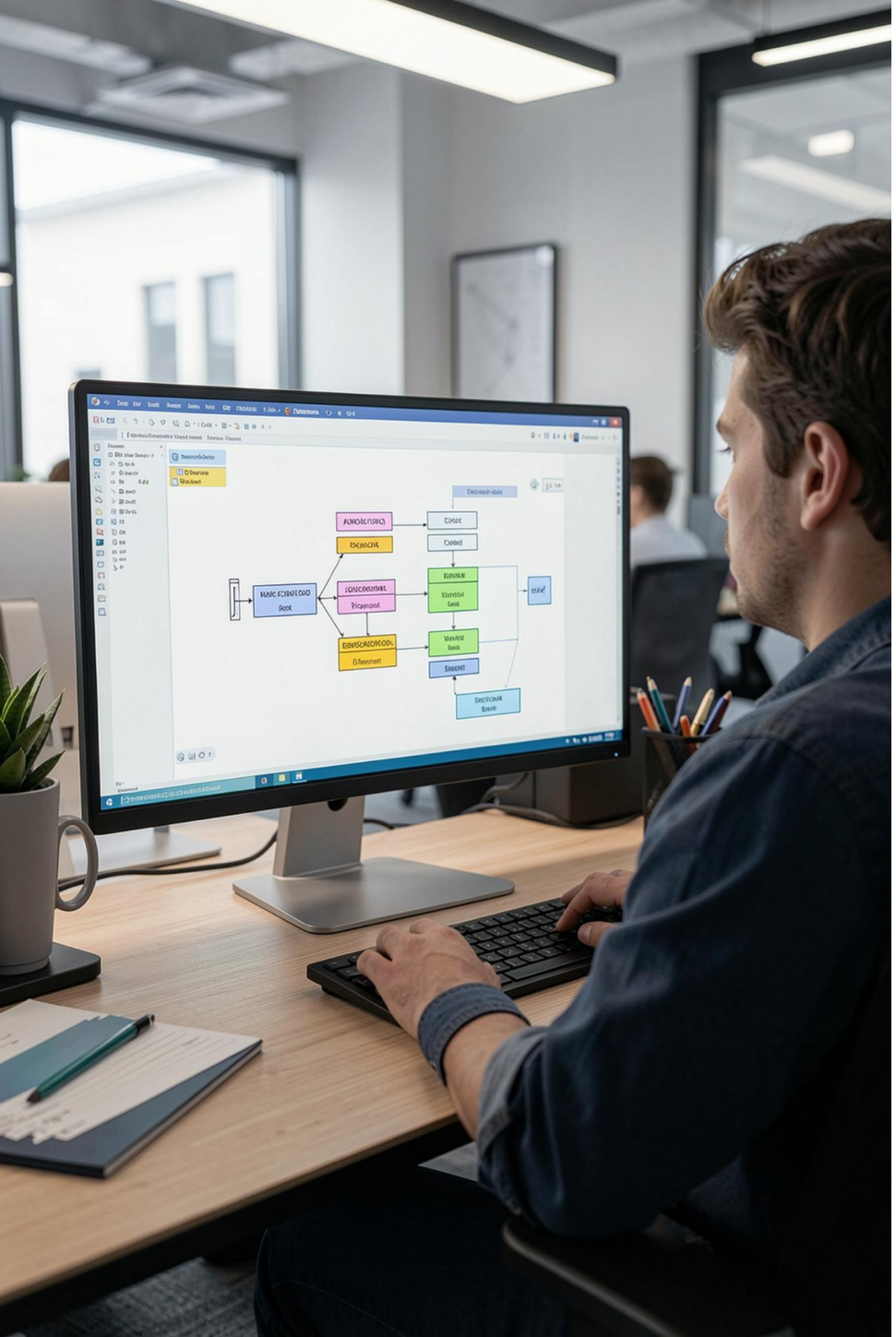
UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN



> Un modelo es una **abstracción de un sistema** que permite ocuparnos en los detalles relevantes para un propósito específico.

¿Por qué modelar?

- La abstracción permite enfocarse en lo relevante
- Herramienta fundamental para abordar sistemas complejos
- Técnica ampliamente empleada en ingeniería de software



¿Qué Ofrece el Modelado?

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

Visualizar un Sistema

Representar gráficamente la estructura y comportamiento del software

Especificar Comportamiento

Definir con precisión cómo debe actuar el sistema ante distintas situaciones

Crear Plantillas

Guías que orientan al equipo durante todo el proceso de desarrollo

Documentar Decisiones

Registrar las decisiones de diseño tomadas a lo largo del proyecto

¿Qué Lenguaje Utilizar para Modelar Software?

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

Código Fuente

Difícil de entender y procesar — El código fuente es demasiado detallado y técnico para comunicar ideas de diseño de alto nivel entre todos los miembros del equipo.

Lenguaje Natural

Propenso a errores — El lenguaje natural es ambiguo e impreciso, lo que puede generar malentendidos en los requerimientos y el diseño del sistema.

Lenguaje Visual ✓

Fácil de interpretar y procesar — Los diagramas y notaciones gráficas como UML permiten comunicar ideas complejas de forma clara, precisa y universal.

El lenguaje visual es la opción preferida para el modelado de software por su claridad y universalidad.

Arquitectura de Software

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

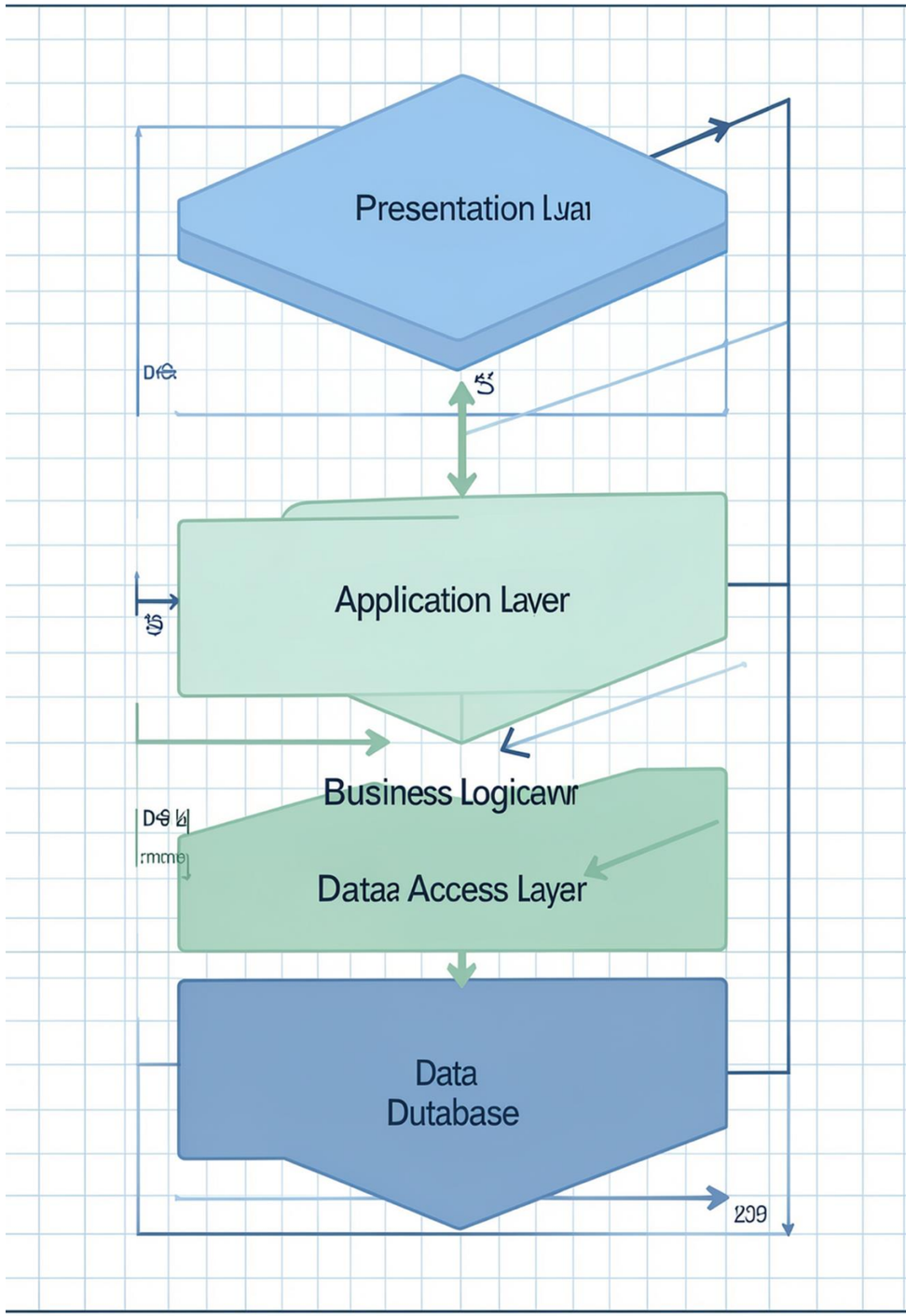
> Conjunto de patrones que proporcionan un **marco de referencia** necesario para guiar la construcción de un software.

Marco Compartido

Permite a programadores, analistas y desarrolladores compartir una misma línea de trabajo y cubrir todos los objetivos y restricciones de la aplicación.

Nivel Más Alto de Diseño

Establece la estructura, funcionamiento e interacción entre las partes del software, siendo el nivel más alto en el diseño de la arquitectura de un sistema.



Arquitectura de Software — Rol Estratégico

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN



La Columna Vertebral del Sistema

La arquitectura de software forma la **columna vertebral** para construir un sistema de software. Es en gran medida responsable de permitir o no ciertos atributos de calidad del sistema, entre los que se destacan:

Además, es un **modelo abstracto reutilizable** que puede ser aplicado en diferentes contextos de desarrollo.

Confiabilidad

Fiabilidad del sistema en producción

Rendimiento

Eficiencia y velocidad de respuesta

Método y Notación

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

Notación de Diseño

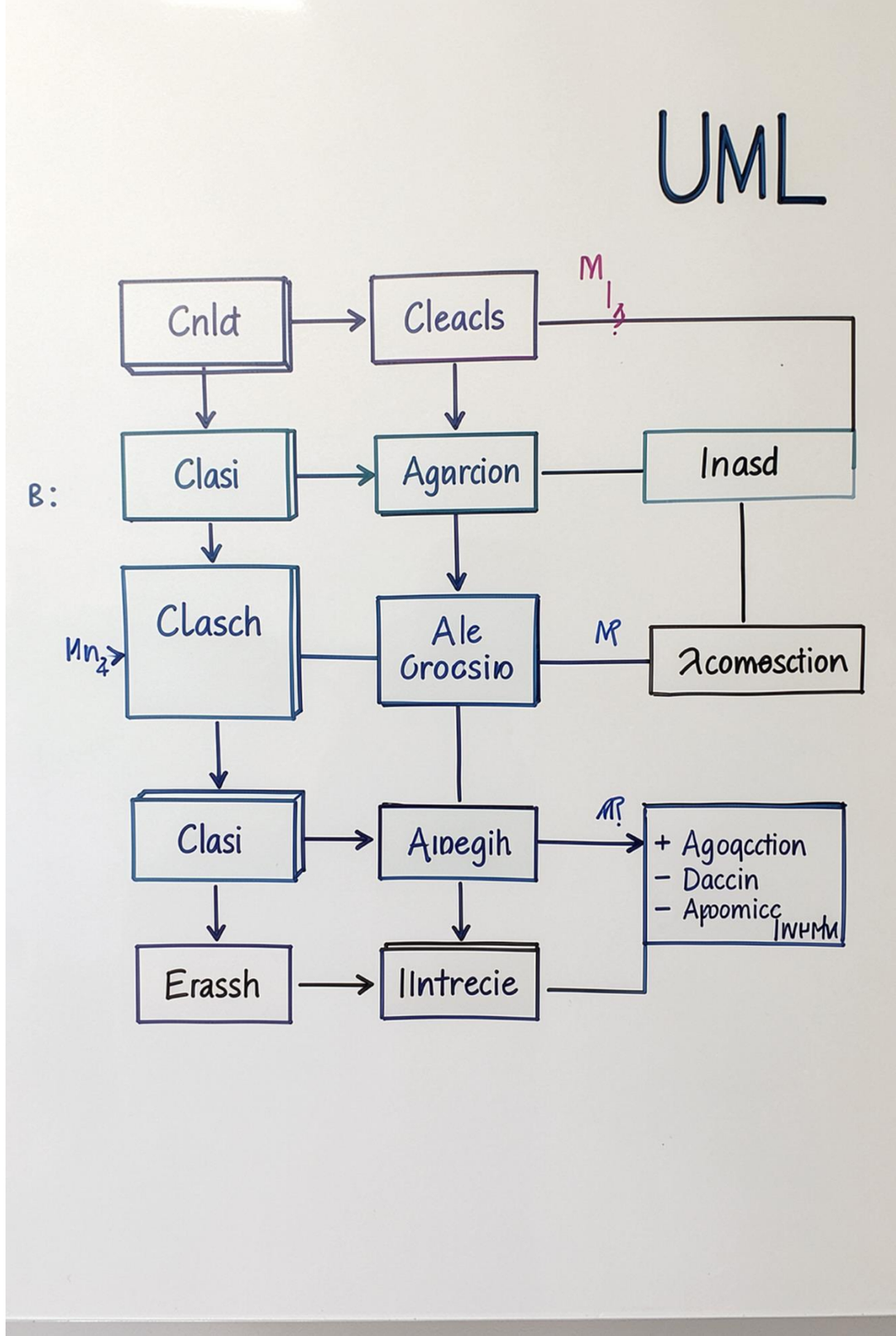
Medio para describir un diseño de software, ya sea **gráfica, textualmente, o ambas**. Los diagramas de clases son notación gráfica; el pseudocódigo es notación textual.

Concepto de Diseño

Idea fundamental que se puede aplicar al diseño de un sistema. UML es una notación gráfica para aplicaciones orientadas a objetos.

Estrategia de Diseño

Plan general y dirección para desarrollar un diseño de software de manera sistemática y coherente.



Método y Notación — Criterios y Métodos

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

Criterios de Estructuración

Guías utilizadas para ayudar a un diseñador a estructurar un sistema de software en sus componentes, definiendo cómo se organizan y relacionan entre sí.

Método de Diseño de Software

Enfoque sistemático que describe la secuencia de pasos a seguir para crear un diseño, dados los requisitos de software de la aplicación. Ayuda al diseñador a identificar las decisiones de diseño que se deben tomar y el orden en que se deben tomar.

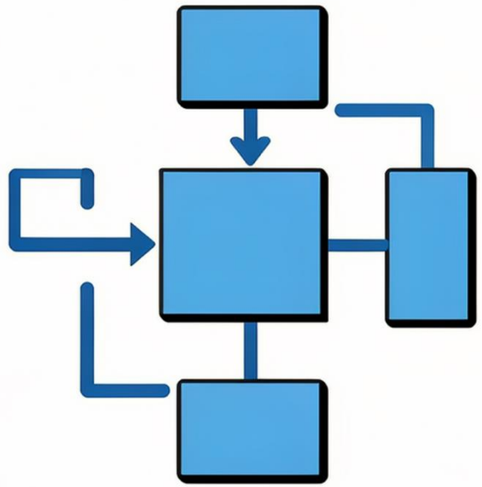
Base del Método

Un método de diseño se basa en un conjunto de **conceptos de diseño**, emplea una o más **estrategias de diseño** y documenta el diseño resultante utilizando una **notación de diseño**.

El Estándar UML

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

UML son las siglas de **"Unified Modeling Language"** o **"Lenguaje Unificado de Modelado"**. Se trata de un estándar adoptado a nivel internacional por numerosos organismos y empresas para crear esquemas, diagramas y documentación relativa a los desarrollos de software.



Estándar Internacional

Adoptado por organismos y empresas a nivel global

Esquemas y Diagramas

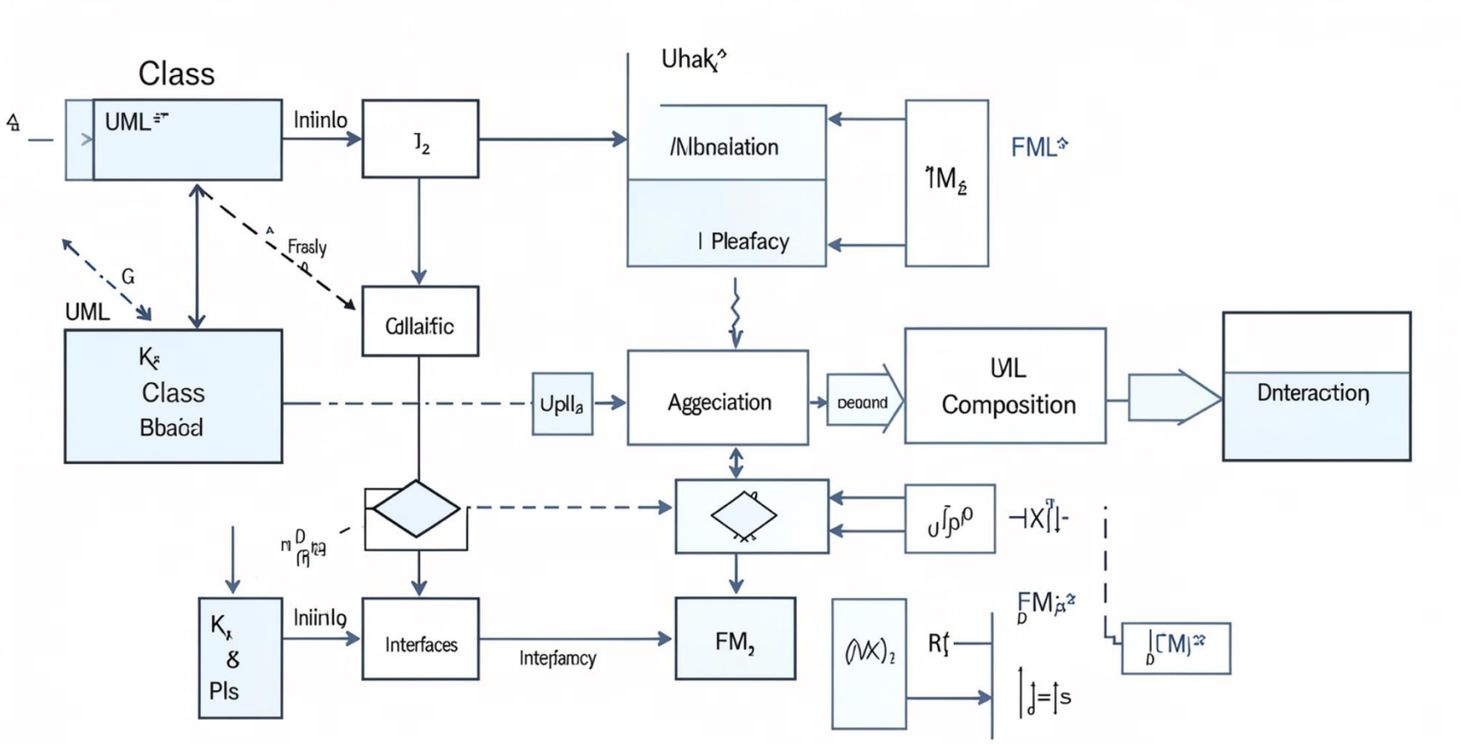
Representación visual de sistemas de software

Documentación

Soporte para toda la documentación del desarrollo

El Estándar UML — ¿Qué es?

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN



Aclarando el Concepto

El término **"lenguaje"** ha generado bastante confusión respecto a lo que es UML. En realidad, el término lenguaje quizás no es el más apropiado, ya que no es un lenguaje propiamente dicho.

UML es una serie de **normas y estándares gráficos** respecto a cómo se deben representar los esquemas relativos al software, no un lenguaje de programación.

Es una herramienta de **comunicación y documentación** que permite describir sistemas con mayor o menor detalle.

El Estándar UML — Definición Práctica

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

> UML es un **set de diagramas** que nos permiten describir nuestro software con mayor o menor detalle y que sirve como **herramienta de comunicación** en nuestros equipos.



Set de Diagramas

Conjunto estructurado de representaciones visuales del sistema



Nivel de Detalle Variable

Permite describir el software con mayor o menor granularidad según la necesidad



Herramienta de Comunicación

Facilita el entendimiento compartido dentro del equipo de desarrollo

UML — Diagramas de Estructura

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

Diagrama de Clases

Describe los diferentes tipos de objetos en un sistema y las relaciones existentes entre ellos. Muestra propiedades, operaciones y restricciones de conexiones entre objetos.

Diagrama de Objetos

También llamado Diagrama de Instancias. Foto de los objetos en un sistema en un momento del tiempo.

Diagrama de Paquetes

Muestra la estructura y dependencia entre paquetes, los cuales permiten agrupar elementos para la descripción de grandes sistemas.

Diagrama de Despliegue

Muestra la relación entre componentes o subsistemas software y el hardware donde se despliega o instala.

Diagrama de Estructura Compuesta

Descompone jerárquicamente una clase mostrando su estructura interna.

Diagrama de Componentes

Muestra la jerarquía y relaciones entre componentes de un sistema software.

UML — Diagramas de Comportamiento

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

Diagramas Principales

Casos de Uso

Captura los requerimientos funcionales de un sistema.

Diagrama de Estado

Muestra el comportamiento de un objeto a lo largo de su vida.

Diagrama de Actividad

Describe la lógica de un procedimiento, proceso de negocio o workflow.

Diagramas de Interacción

Diagrama de Secuencia

Muestra los mensajes pasados entre objetos en un escenario.

Diagrama de Comunicación

Muestra interacciones entre participantes con énfasis en la secuencia de mensajes.

Diagrama de Tiempo

Foco en las restricciones temporales de un objeto o conjunto de objetos.

Diagrama de Colaboración

Muestra las interacciones organizadas alrededor de los roles (UML 1.X).

Evolución del Modelado de Software

HISETOFYG

Eneflotlute Programming Engineming

FILODRIOIN
OP SOFTWARE
ENGINEERING

UNIDAD I · SOFTWARE: VISIÓN GENERAL · INTRODUCCIÓN

1

Década de 1960

Programas con poco o ningún análisis sistemático. Uso de diagramas de flujo. Las subrutinas como medio de compartir bloques de código.

2

Mediados de 1970

Dos estrategias ganan prominencia: **flujo de datos de diseño orientado** y **diseño estructurado de datos**. Módulos como herramienta de gestión de proyectos.

3

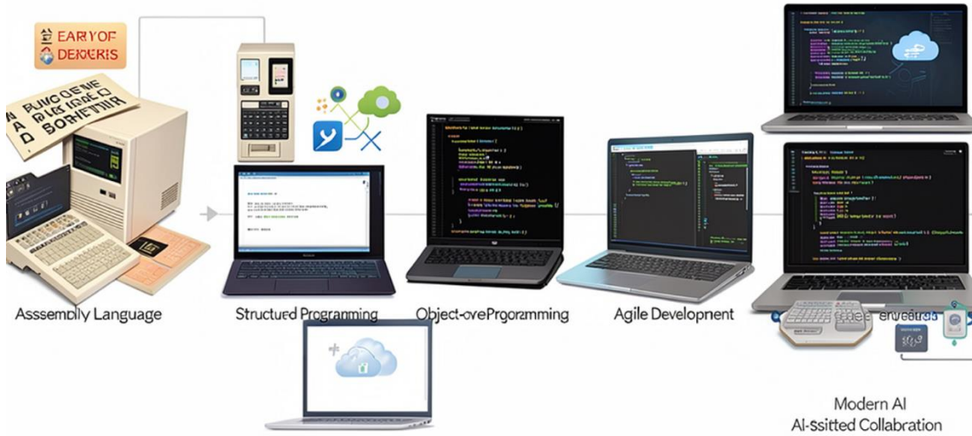
Década de 1980

Maduración general de los métodos de diseño de software. Introducción de varios métodos de diseño de sistemas.

4

Finales de 1980

Popularidad de la **programación orientada a objetos**. Énfasis en modelar el dominio del problema, ocultamiento de información y herencia.



Contenido del Curso — Resumen

Unidad I

Software: Visión General

- Introducción al modelado

Unidad II

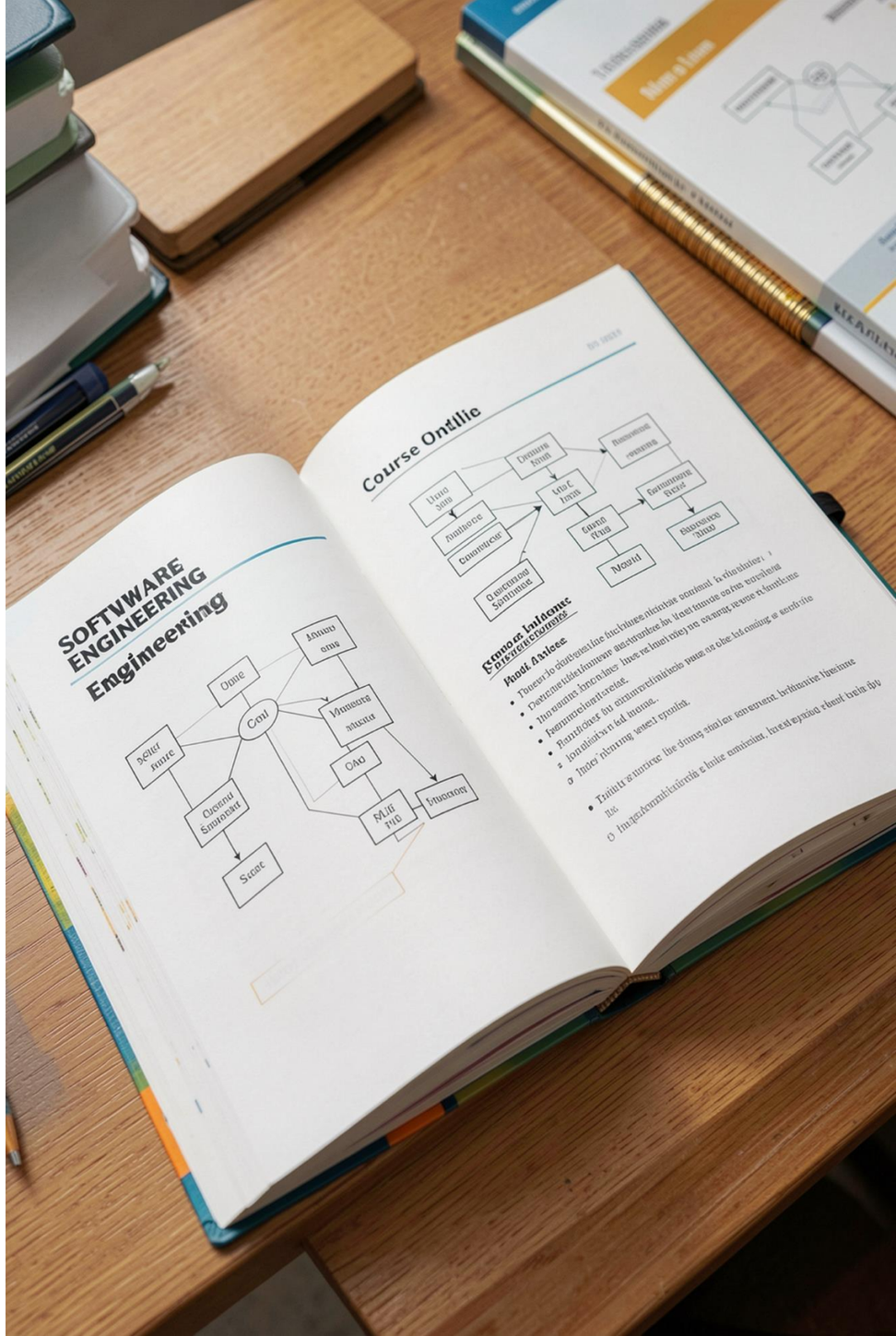
Modelado de Software

- Modelando casos de uso
- Notación UML

Unidad III

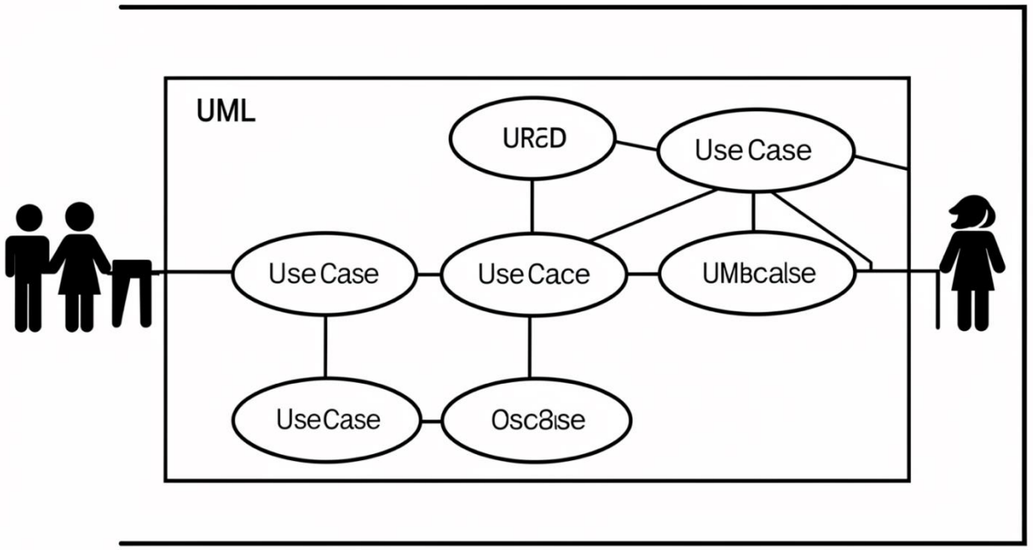
Proyectos, Metodologías y Marcos de Trabajo

- Ciclo de Vida del Software
- Diseño y Arquitectura
- Metodologías Ágiles



Modelando Casos de Uso — Contenido

UNIDAD II · MODELADO DE SOFTWARE



Temas de la Unidad

01

Modelado de Requerimientos

Fundamentos del proceso de captura de requerimientos

02

Casos de Uso y Actores

Identificación y documentación en el Modelo de Caso de Uso

03

Requerimientos No Funcionales

Restricciones y propiedades emergentes del sistema

ĆÇ

Diagramas de Actividades

Descripción de lógica de procesos y workflows

Unidad II — Modelado de Software

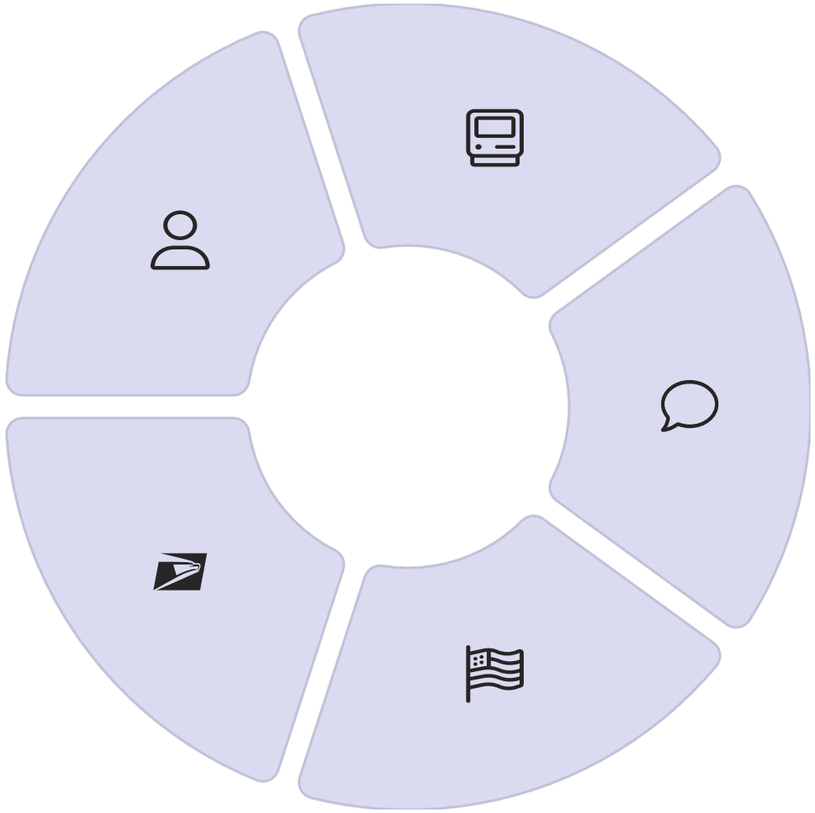
NOTACIÓN UML

Casos de Uso

Captura de requerimientos funcionales

Packages

Organización de grandes sistemas



Clases y Objetos

Estructura estática del sistema

Interacción

Colaboración entre objetos

State Machine

Comportamiento dinámico

Requerimientos de Software

UNIDAD II · MODELANDO CASOS DE USO



¿Qué son los Requerimientos?

Son las **descripciones de los servicios** que debe proveer el sistema y las restricciones bajo las cuales debe operar.

Ingeniería de Requerimientos

Es el proceso de **descubrir, analizar, documentar y verificar** estos servicios y restricciones.

⚠ Comprender la naturaleza de los problemas puede ser muy difícil, especialmente si el dominio es nuevo para el equipo.

El Término "Requerimiento"

REQUERIMIENTOS DE SOFTWARE

El término **requerimiento** no se utiliza de forma consistente en la industria del software.

Como Declaración de Alto Nivel

Un requerimiento se visualiza como una **declaración abstracta de alto nivel** de un servicio que debe proveer el sistema, expresada en términos comprensibles para el cliente.

Como Restricción del Sistema

Un requerimiento también puede entenderse como una **restricción** del sistema, definiendo los límites dentro de los cuales debe operar el software.

Esta dualidad en el uso del término genera ambigüedad en los proyectos. Es fundamental establecer una definición clara desde el inicio.

Documento de Requerimientos para el Sistema

REQUERIMIENTOS DE SOFTWARE

La Compañía (Cliente) debe:

- Definir sus necesidades de forma suficientemente abstracta
- Redactar requerimientos que permitan a varios contratistas licitar el contrato
- Ofrecer formas diferentes de cumplir las necesidades de los clientes

El Contratista (Desarrollador) debe:

- Redactar una definición del sistema para el cliente
- Asegurar que el cliente comprenda y pueda validar lo que hará el software
- Producir el "documento de requerimientos para el sistema"



Requerimientos del Usuario vs. del Sistema

REQUERIMIENTOS DE SOFTWARE



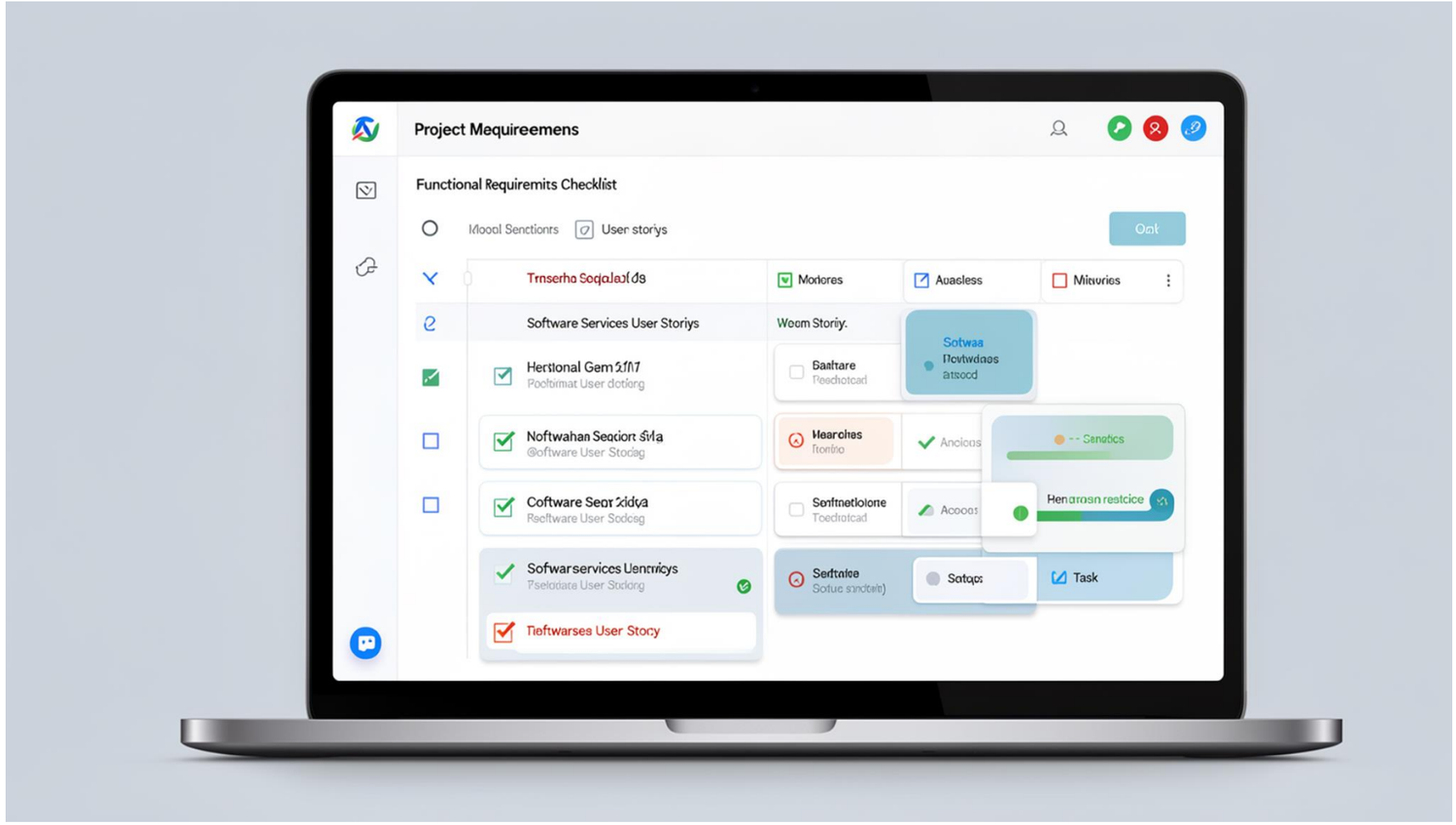
Dos Niveles de Especificación

Los **requerimientos del usuario** son declaraciones en lenguaje natural y diagramas de los servicios que se espera que el sistema provea y de las restricciones bajo las cuales debe operar.

Los **requerimientos del sistema** establecen con detalle los servicios y restricciones del sistema. El documento de requerimientos del sistema, denominado **Especificación Funcional**, debe ser preciso y sirve como un contrato entre el comprador y el desarrollador.

Tipos de Requerimientos

REQUERIMIENTOS DE SOFTWARE



Requerimientos Funcionales

Son declaraciones de los **servicios que proveerá el sistema**, de la manera en que éste reaccionará en situaciones particulares. Describen qué debe hacer el sistema.

Requerimientos No Funcionales

Son **restricciones de los servicios o funciones** ofrecidos por el sistema. Incluyen restricciones de tiempo, sobre el proceso de desarrollo, estándares, etc. Describen cómo debe comportarse el sistema.



Requerimientos No Funcionales

REQUERIMIENTOS DE SOFTWARE

Propiedades Emergentes

No se refieren directamente a las funciones específicas que entrega el sistema, sino a las **propiedades emergentes** de éste como la fiabilidad, la respuesta en el tiempo y la capacidad de almacenamiento.

Criticidad

A menudo son **más críticos que los requerimientos funcionales** particulares. Una falla en un requerimiento no funcional del sistema puede inutilizarlo por completo.

Requerimientos No Funcionales — Origen

REQUERIMIENTOS DE SOFTWARE



Más Allá del Software

Los requerimientos no funcionales no siempre se refieren al sistema de software a desarrollar. Algunos de estos requerimientos **restringen el proceso** a utilizar en el desarrollo del sistema.

Origen de los Requerimientos No Funcionales

- Necesidades del usuario
- Restricciones en el presupuesto
- Políticas organizacionales
- Necesidad de interoperabilidad con otros sistemas
- Factores externos como regulaciones y legislación

Clasificación de Requerimientos No Funcionales

REQUERIMIENTOS NO FUNCIONALES



Los requerimientos no funcionales se clasifican en tres grandes categorías que abarcan desde las propiedades internas del producto hasta las restricciones externas impuestas por el entorno organizacional y legal.



Requerimientos de Software — Síntesis

REQUERIMIENTOS DE SOFTWARE

2

Tipos Principales

Funcionales y No Funcionales

4

Etapas de la IR

Descubrir, Analizar, Documentar, Verificar

2

Niveles de Especificación

Usuario y Sistema

MyD 2026 · Ing. Fernando Bono

Proceso de Ingeniería de Requerimientos

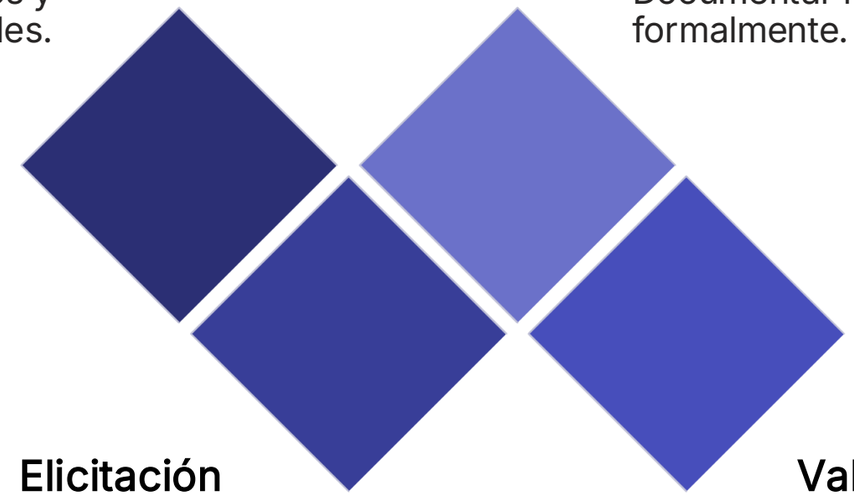
REQUERIMIENTOS DE SOFTWARE

Análisis

Identificar conflictos y prioridades.

Especificación

Documentar requerimientos formalmente.



Elicitación

Descubrir necesidades con stakeholders.

Validación

Verificar que los requerimientos sean correctos.

La Ingeniería de Requerimientos es un proceso iterativo que busca garantizar que el sistema a desarrollar satisfaga las necesidades reales de los usuarios y la organización. Cada etapa retroalimenta a las anteriores.

Requerimientos No Funcionales — Ejemplos

REQUERIMIENTOS NO FUNCIONALES



¿Por qué son Críticos?

Los requerimientos no funcionales definen la **calidad percibida** del sistema. Un sistema que funciona correctamente pero es lento, inseguro o difícil de usar, será rechazado por los usuarios.

⊗ Una falla en un requerimiento no funcional puede inutilizar el sistema completo, incluso si todos los requerimientos funcionales están implementados correctamente.

Métricas de Calidad

- Velocidad de procesamiento
- Tasa de fallos del sistema
- Tiempo de recuperación
- Facilidad de uso medida

¿Preguntas?

Modelación y Diseño de Sistemas 2026

Ing. Fernando Bono

linkedin.com/in/fernandobono/

Próxima Clase

Profundizaremos en el modelado de casos de uso, identificación de actores y la notación UML para representar requerimientos funcionales del sistema.

